

FICHE TECHNIQUE

Serveur IA Local & Détection de Plagiat

SAE204 — Soutenance Cybersécurité

Auteur	Enzo Maresia
Formation	BUT Réseaux & Télécommunications — Auxerre
GitHub	AYK0-prog/Local_AI_server
Date	Juin 2026
Objet	Référence technique pour la soutenance orale

Ce document explique le fonctionnement technique du projet, les décisions prises et les réponses aux questions susceptibles d'être posées en soutenance.

1. Vue d'ensemble du système

Le projet réalise deux choses distinctes :

Bloc	Ce que ça fait	Comment
IA locale	Héberger un assistant conversationnel sans Internet	Ollama + OpenWebUI dans Docker
Détection plagiat	Comparer deux rapports PDF et produire un score de risque	Pipeline Python en 8 étapes + interface Flask

2. Matériel et réseau

Composant	Détail	Rôle dans le projet
2× Xeon E5-2630 v3	32 cœurs logiques au total	Calcul CPU : embeddings, OCR, Flask
RTX 3060 Ti	8 Go VRAM	Inférence GPU : Ollama (gemma3, llama...)
32 Go DDR3	RAM système	Modèles en RAM si VRAM insuffisante
Réseau	172.17.0.0/24 — salle réseaux IUT	Réseau privé avec accès Internet, connexion filaire

3. Les briques logicielles — à quoi sert chacune ?

Outil	Rôle simple	Pourquoi ce choix ?
Ollama	Serveur local de LLM — télécharge les modèles et répond aux requêtes via une API REST (port 11434)	Simple à installer, supporte le GPU, API compatible OpenAI
OpenWebUI	Interface web type ChatGPT branchée sur Ollama	Multi-comptes, headless, déployé en Docker en 1 commande
Docker	Isole OpenWebUI du système dans un conteneur	Isolation propre, redémarrage automatique, pas de conflits de dépendances
Flask	Serveur web Python qui expose l'interface de détection de plagiat	Léger, intégré nativement avec le pipeline Python
WeasyPrint	Génère le rapport PDF final depuis un template HTML	Permet un rapport propre et téléchargeable depuis le navigateur

4. Le pipeline de détection — les 8 étapes expliquées

Quand on soumet deux rapports PDF, le programme exécute ces 8 étapes dans l'ordre. Chaque étape produit un score partiel. Le score final est une moyenne pondérée.

1	Extraction PDF → texte PyMuPDF + Tesseract OCR	PyMuPDF lit le PDF page par page. Si une page contient du texte natif, il l'extrait directement. Si c'est une image scannée (photo de TP), Tesseract (moteur OCR) lit les caractères et les convertit en texte. Le résultat est un fichier Markdown lisible.
2	Exclusion du sujet Shingling (filtre)	Si un énoncé PDF est fourni (argument --sujet), on retire du texte toutes les phrases qui proviennent de l'énoncé. Ça évite de signaler comme plagiat des formulations que tous les élèves ont copiées de l'énoncé.
3	Détection lexicale ×35 % Shingling k=5 + rapidfuzz	On découpe chaque texte en "shingles" : séquences de 5 mots consécutifs. Ex : "le réseau local permet la" est un shingle. On compare les deux ensembles de shingles avec rapidfuzz (similarité floue, tolérante aux fautes de frappe). Plus deux textes partagent de shingles, plus le score lexical est élevé.
4	Détection sémantique ×35 % Embeddings multilingual-e5-base + cosinus	On transforme chaque phrase en un vecteur de nombres (embedding). Deux phrases avec le même sens — même si les mots sont différents — auront des vecteurs proches. On mesure cette proximité avec la similarité cosinus. Si le cosinus est ≥ 0.85 , les phrases sont considérées sémantiquement identiques. C'est ce qui détecte la PARAPHRASE, invisible au signal lexical.
5	Analyse d'images ×15 % Hash perceptuel (imagehash)	Pour chaque image extraite des PDFs, on calcule un "hash perceptuel" : une empreinte numérique de l'image, robuste aux petites modifications (compression, recadrage léger). Si la distance entre deux hashes est ≤ 10 , les images sont considérées identiques. Note : LLaVA (analyse IA des images) est désactivé — contrainte VRAM 8 Go.
6	Verdict IA local ×15 % gemma3:4b via Ollama	On envoie à Ollama (gemma3:4b) un résumé des résultats des étapes 3, 4 et 5. Le modèle analyse le contexte et retourne un verdict structuré en JSON avec un champ "plagiat", un champ "confiance" et une "justification" en langage naturel. Ce score est pondéré par l'indice de confiance du modèle.
7	Score final & niveau de risque Agrégation pondérée	Score = (lexical $\times 0.35$) + (sémantique $\times 0.35$) + (images $\times 0.15$) + (IA $\times 0.15$). Niveau FAIBLE si $< 40\%$, MODÉRÉ entre 40% et 69% , ÉLEVÉ si $\geq 70\%$.
8	Génération du rapport Markdown → HTML → PDF (WeasyPrint)	Le pipeline formate les résultats en Markdown, les convertit en HTML stylisé, puis WeasyPrint génère un rapport PDF téléchargeable depuis l'interface Flask.

5. Les modèles IA — rôle de chacun

Modèle	Type	Où il est utilisé	Pourquoi ce modèle ?
gemma3:4b	LLM texte	Étape 6 — verdict IA du pipeline	Léger (~3 Go VRAM), rapide, bon pour du JSON structuré
qwen2.5:7b	LLM texte	OpenWebUI — assistant nommé du CDC	Multilingue FR/EN, performant pour la conversation
llama3.2:3b	LLM texte	Tests rapides, usage courant	Très léger (~2 Go), idéal pour les prototypes
llama3.1:8b	LLM texte	Second avis IA (remplace la vérif. externe)	Plus puissant que gemma3:4b pour le raisonnement
llava:7b	Vision (VLM)	INSTALLÉ mais DÉSACTIVÉ	Désactivé : boucle CPU à 1100 % (encodeur CLIP) + VRAM insuffisante
multilingual-e5-base	Embeddings	Étape 4 — similarité sémantique, forcé sur CPU	Modèle multilingue FR/EN, optimisé pour la comparaison de phrases

6. Comment fonctionne l'interface Flask ?

L'application Flask (port 5000) est une interface web qui pilote le pipeline CLI sans que l'enseignant ait besoin d'utiliser un terminal. Voici le flux complet :

1. Login	L'enseignant se connecte sur <code>http://localhost:5000</code> (admin/admin).
2. Upload	Il dépose les 2 rapports PDF à comparer (+ l'énoncé en option).
3. Lancement	Flask appelle la fonction <code>analyze()</code> du pipeline via les liens symboliques.
4. Terminal live	Flask intercepte les <code>print()</code> du pipeline (monkey-patching de <code>sys.stdout</code>) et envoie chaque ligne au navigateur.
5. Rapport	À la fin, le rapport PDF est généré et accessible depuis l'interface. L'analyse est enregistrée dans la base SQL.

7. Contraintes techniques & décisions prises

Contrainte	Symptôme rencontré	Décision prise
8 Go VRAM — <code>sentence_transformers</code> sur GPU	Ollama détectait <code>limited-vram</code> , offloadait LLaVA sur CPU	<code>sentence_transformers</code> forcé sur CPU (<code>device="cpu"</code>)
8 Go VRAM — LLaVA + Ollama simultanés	llama-server (encodeur CLIP) bouclait à 1100 % CPU	LLaVA désactivé (<code>vision_model = None</code>), hash perceptuel uniquement
Xeon E5-2630 v3 sans AVX2	LM Studio refusait de démarrer	LM Studio abandonné — Ollama retenu (pas d'exigence AVX2)

Clé HuggingFace invalide (401)	Vérification externe impossible	Remplacée par llama3.1:8b local — pipeline 100 % local
Ports SSH/VNC bloqués par l'IUT	Impossible de se connecter à distance	Accès physique au serveur uniquement

8. Questions probables en soutenance — réponses préparées

Q C'est quoi Ollama exactement ?

→ Ollama est un serveur local qui télécharge des modèles de langage (LLM) depuis Internet et les exécute sur la machine. Il expose une API REST sur le port 11434. Quand le pipeline envoie une requête à `http://localhost:11434`, Ollama charge le modèle en VRAM et retourne la réponse. Tout reste sur notre machine, rien ne sort sur Internet.

Q Pourquoi faire de la détection en local plutôt qu'avec une API comme ChatGPT ?

→ Pour des raisons de confidentialité et de RGPD : les rapports d'élèves sont des données personnelles. Les envoyer à une API externe (OpenAI, Google) pose des problèmes légaux. En local, les données ne quittent jamais le serveur de l'IUT.

Q C'est quoi un embedding ?

→ Un embedding c'est la transformation d'une phrase en un vecteur de nombres (une liste de valeurs numériques). Le modèle multilingual-e5-base fait ça. L'intérêt : deux phrases avec le même sens — même si les mots sont complètement différents — auront des vecteurs proches. On mesure cette proximité avec la similarité cosinus (de 0 à 1). Au-delà de 0.85, on considère que les phrases sont sémantiquement identiques.

Q C'est quoi le shingling ?

→ On découpe le texte en fenêtres glissantes de 5 mots consécutifs qu'on appelle des shingles. Exemple avec $k=5$: 'le réseau local permet la' est un shingle, puis on décale d'un mot pour avoir le suivant. On compare ensuite les ensembles de shingles des deux rapports. Si beaucoup de shingles sont identiques ou très proches (avec rapidfuzz), le score lexical est élevé.

Q Pourquoi LLaVA est-il désactivé ?

→ LLaVA est un modèle de vision qui analyse des images. Le problème : il prend ~4.7 Go de VRAM. En même temps, sentence_transformers chargeait le modèle d'embeddings sur le GPU en prenant ~2.3 Go supplémentaires. La RTX 3060 Ti n'a que 8 Go de VRAM : il n'en restait plus assez pour Ollama. Résultat : llama-server bouclait à 1100 % CPU. On a forcé sentence_transformers sur CPU et désactivé LLaVA. L'analyse d'images repose maintenant uniquement sur le hash perceptuel.

Q C'est quoi le hash perceptuel ?

→ C'est une empreinte numérique d'une image, calculée à partir de ses grandes structures visuelles (luminosité, contrastes). Contrairement à un hash MD5 classique, il est robuste aux petites modifications : compresser une image ou la recadrer légèrement donnera un hash perceptuel très proche. Si la distance entre deux hashes est ≤ 10 , on considère les images comme identiques.

Q Pourquoi Docker pour OpenWebUI mais pas pour Ollama ?

→ Ollama doit accéder directement au GPU de la machine hôte pour ses calculs. C'est possible dans Docker avec nvidia-container-toolkit, mais ça ajoute de la complexité. On a choisi d'installer Ollama directement sur l'hôte pour simplicité et performance. OpenWebUI, lui, est une application web sans besoin GPU : Docker l'isole proprement du système sans inconvénient.

Q C'est quoi les SSE (Server-Sent Events) ?

→ C'est un mécanisme HTTP où le serveur garde une connexion ouverte avec le navigateur et envoie des données au fil de l'eau, sans que le navigateur ait à redemander. On l'utilise pour le terminal live : Flask intercepte les `print()` du pipeline (monkey-patching de `sys.stdout`), puis pousse chaque ligne vers le navigateur via SSE. Le résultat : l'enseignant voit les logs défiler en temps réel sans rafraîchir la page.

Q Pourquoi plusieurs modèles de langage ?

→ Chaque modèle a un profil différent. `gemma3:4b` est léger et rapide : parfait pour le verdict IA en JSON. `qwen2.5:7b` est multilingue et bon en conversation : c'est l'assistant 'nommé' du cahier des charges. `llama3.1:8b` est plus puissant : il sert de second avis pour valider le verdict de `gemma3`. `llama3.2:3b` est ultra-léger pour les tests rapides. On ne charge qu'un modèle à la fois en VRAM — Ollama fait le swap automatiquement.

Q Pourquoi un sous-réseau privé et non le Wi-Fi de l'IUT ?

→ Le Wi-Fi de l'IUT isole les clients entre eux (isolation client Wi-Fi) : deux machines sur le même réseau Wi-Fi ne peuvent pas se voir. En début de projet, on a contourné ça avec un switch physique et un sous-réseau `10.42.0.0/24`. Maintenant le serveur est directement sur le réseau filaire de la salle réseaux (`172.17.0.0/24`) qui permet la communication entre machines et l'accès Internet.

9. Lexique technique — 30 secondes par terme

Terme	En une phrase
LLM	Large Language Model — modèle de texte entraîné sur des milliards de mots.
VLM	Vision-Language Model — LLM qui comprend aussi les images (ex. LLaVA).
API REST	Interface de communication via HTTP — on envoie une requête, on reçoit une réponse JSON.
VRAM	Mémoire du GPU. Les LLMs y sont chargés pour être calculés rapidement.
Docker	Système de conteneurs : isole une application dans une boîte autonome.
OCR	Optical Character Recognition — lit le texte dans une image scannée.
Cosinus (similarité)	Mesure l'angle entre deux vecteurs. 1 = identiques, 0 = sans rapport.
Headless	Sans interface graphique — le serveur est piloté à distance via réseau.
AVX2	Jeu d'instructions CPU pour accélérer les calculs. Absent sur nos Xeon E5-2630 v3.
WSGI	Interface Python pour serveur de production (gunicorn) — remplace le serveur de dev Flask.
SHA-256	Algorithme de hachage — transforme un mot de passe en empreinte irréversible.
SSE	Server-Sent Events — flux HTTP unidirectionnel serveur → navigateur pour le terminal live.

10. Commandes de référence rapide

Démarrer le pipeline CLI :

```
cd /home/enzo/Téléchargements/SAE204_scripts && source venv/bin/activate
python3 main.py --a A.pdf --b B.pdf --sujet sujet.pdf --output resultats/
```

Démarrer l'application Flask :

```
cd /home/enzo/Téléchargements/scripts && source venv/bin/activate
pkill -f "python app.py" 2>/dev/null && python app.py
```

Vérifications rapides :

```
ollama list # modèles disponibles
nvidia-smi # état du GPU et VRAM
docker ps # conteneurs OpenWebUI actifs
ls -l scripts/modules # vérifier les liens symboliques
```

11. Ports et accès rapides

Service	URL	Identifiants
OpenWebUI (assistant IA)	http://localhost:3000	ps.enzo.maresia@gmail.com / AI_serv

Flask (détection plagiat)	http://localhost:5000	admin / admin
Ollama API	http://localhost:11434	— (accès libre en local)

Fiche technique générée pour la soutenance SAE204 — Enzo Maresia — Juin 2026